

HDB Flat Resale Price Prediction in Singapore: A Data Mining Approach (Level16)

Tan Guan Ming
A0173210X
e0202780@u.nus.edu

Wan Chunwei
A0177345X
e0253705@u.nus.edu

Keshav Nath
A0298325U
e1352463@u.nus.edu

Yang Aodi
A0304400A
e1373488@u.nus.edu

Abstract—We predict HDB resale prices in Singapore with a leak-safe tabular pipeline built on gradient-boosted trees. Deterministic preprocessing standardises schema/text and isolates train/test; features include temporal and lease signals, town/type/model interactions, a quarter-aligned Resale Price Index (RPI), and amenity distances. Using 5-fold cross-validation and Optuna, tuned CatBoost and XGBoost achieve RMSEs of 25,115 and 25,382 SGD; a simple 0.5/0.5 blend reaches 24,673 SGD (59.6% vs. a numeric-only baseline at 61,036). Scores are stable across folds (0.3%), yielding a compact and reproducible approach for mixed tabular price prediction.

Index Terms—HDB resale price prediction, gradient boosting, CatBoost, XGBoost, feature engineering, cross-validation

Github Repository: CS5228-Project-Level16

I. INTRODUCTION

HDB resale prices are central to Singapore’s housing market, affecting buyers, sellers, and policymakers. While size, flat type, floor level, and proximity to amenities (MRT stations, schools, malls) are known correlates, their quantitative contributions and interactions remain uncertain.

We frame price prediction as supervised regression in Singapore dollars (SGD) using structured property data—town, flat type, floor area, floor range, model, lease commencement year—augmented with amenity proximity and a quarter-aligned Resale Price Index (RPI). Beyond accuracy, we examine underlying data patterns and feature importance. Performance is reported as Root Mean Squared Error (RMSE) on the CS5228 Kaggle contest split [1].

II. EXPLORATORY DATA ANALYSIS

We profile the dataset to characterise distributional shape, temporal structure, property attributes, and geographic effects of HDB resale price. The data include geography (town, street, address), time (month of sale, lease commencement year), property details (flat type, floor area, floor range), and amenity proximity (e.g., nearest MRT / schools / malls).

A. Target distribution

Resale prices are right-skewed with a long upper tail (Figure 1). Because our final models are tree ensembles that are invariant to monotonic transformations of the target, we model raw prices directly rather than applying a log transform. To summarise the distribution robustly we therefore report the median and the interquartile range (IQR), and we complement

mean-based metrics with median-based summaries in the evaluation section.

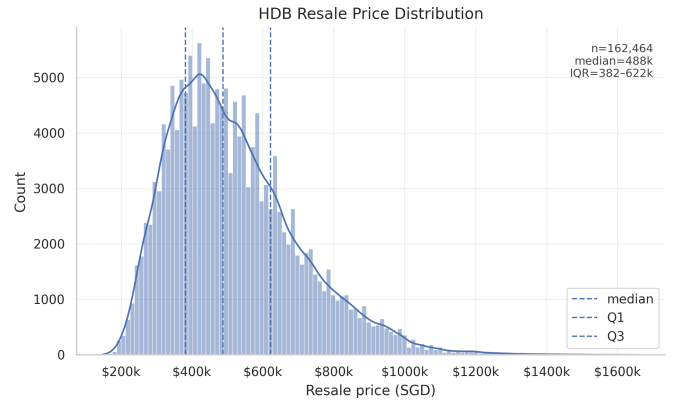


Fig. 1: Resale price distribution (median and IQR marked).

B. Temporal structure

Monthly medians exhibit a persistent upward trend since 2017; Figure 2 reports median with 95% bootstrap CIs and a 12-month moving average. Volatility, measured robustly by IQR, also rises over time (Figure 3). When a macro index is available (e.g., RPI), resale medians co-move with it once both are indexed to a common baseline [2]. (Figure 4), indicating that macro movements dominate seasonal patterns. After detrending, month-of-year effects are small with narrow CIs (Figure 5), suggesting weak pure seasonality relative to the long-run trend. To avoid temporal leakage, all temporal features use only contemporaneous information (e.g., quarter-level RPI), and preprocessing is split-isolated. Model selection uses 5-fold shuffled KFold on the training period (see §Modeling Approach), and final scoring uses the contest test split.

C. Property attributes

Family-sized flats dominate transactions (Figure 6). Here *shares* mean the percentage of all transactions belonging to each flat type, which makes class imbalance explicit. Price rises monotonically with floor area; we summarise the relationship using deciles (median with IQR band) to avoid scatter over-plotting (Figure 7). A premium with floor range is evident even after controlling for location via town-adjusted means and CIs (Figure 8), indicating a genuine storey effect rather than pure geographic confounding.

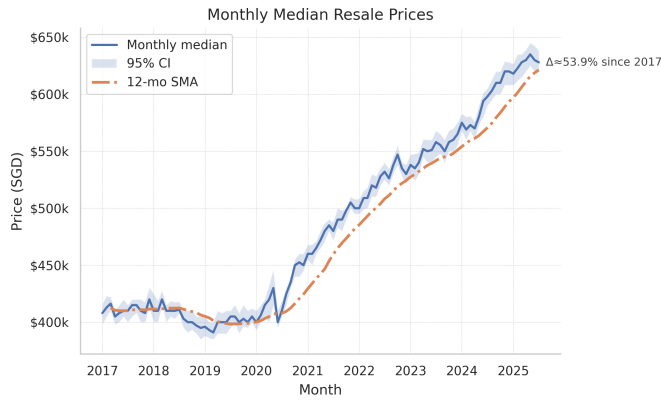


Fig. 2: Monthly median resale price with 95% bootstrap CIs and a 12-month moving average.

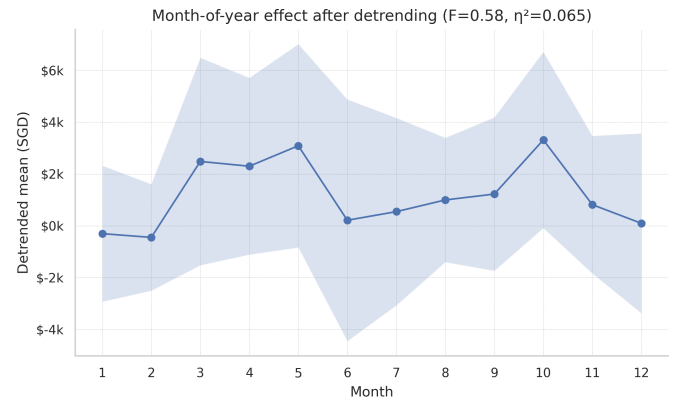


Fig. 5: Month-of-year means after detrending (95% CIs). Seasonality is limited relative to trend.

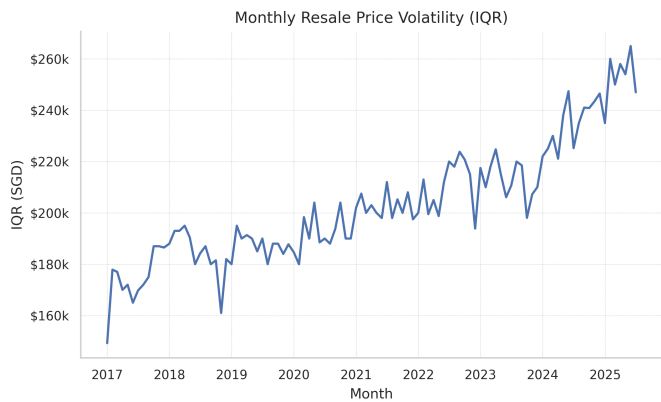


Fig. 3: Monthly volatility (IQR). Dispersion grows in recent years.

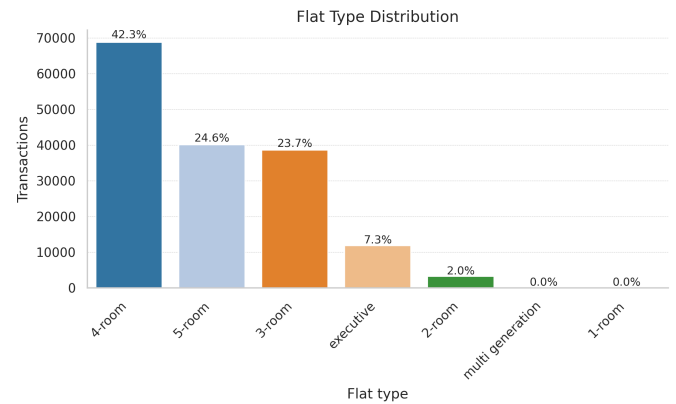


Fig. 6: Flat type distribution (percentage of all sales).

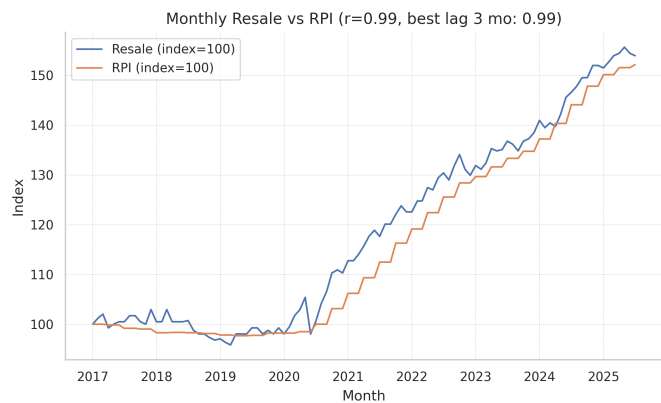


Fig. 4: Resale median vs. RPI, both indexed to 100; title reports correlation and best cross-correlation lag.

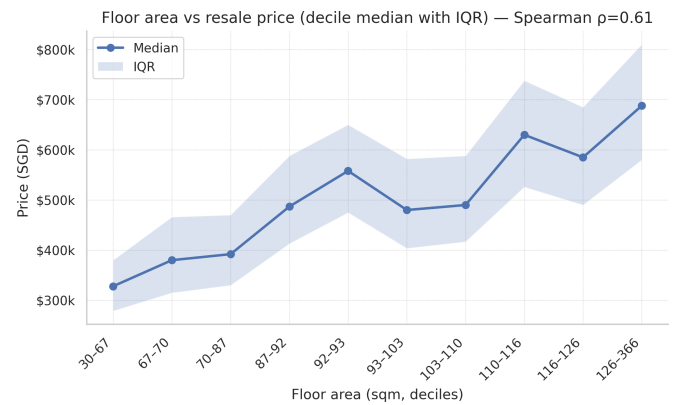


Fig. 7: Floor area vs. price summarised by area deciles (median with IQR).

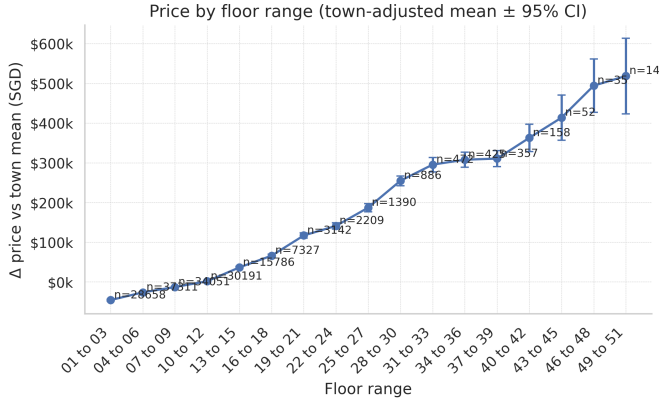


Fig. 8: Town-adjusted mean price by floor range with 95% CIs (ordered by numeric midpoint).

D. Geography and accessibility

Transaction volumes vary sharply by town (Figure 9) and mean prices differ materially (Figure 10); we report 95% CIs and encode sample size to avoid over-interpreting small towns. Amenity *counts* conflate coverage with accessibility, so we use distance-to-nearest features; median distances by town are shown in Figure 11. Distances provide a more stable and predictive measure of accessibility than raw counts.

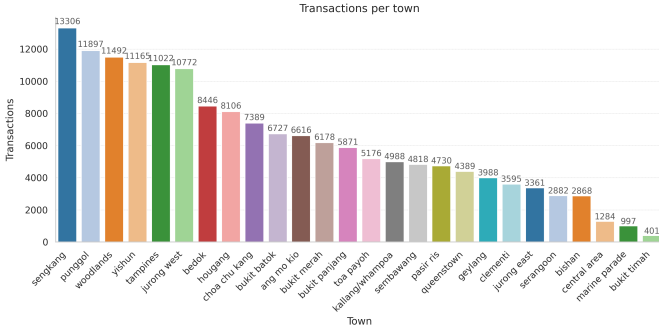


Fig. 9: Transactions per town (ranked).

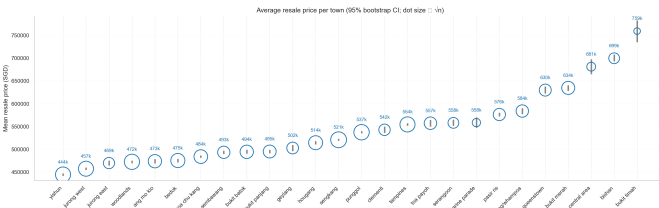


Fig. 10: Average resale price per town with 95% bootstrap confidence intervals. Hollow circle $\propto \sqrt{n}$.

E. Data quality checks

During EDA we observed non-uniformities that would bias encoders and joins if left unaddressed:

- **Inconsistent field names** for lease commencement (LEASE_COMMENCE_DATA vs. LEASE_COMMENCE_DATE).

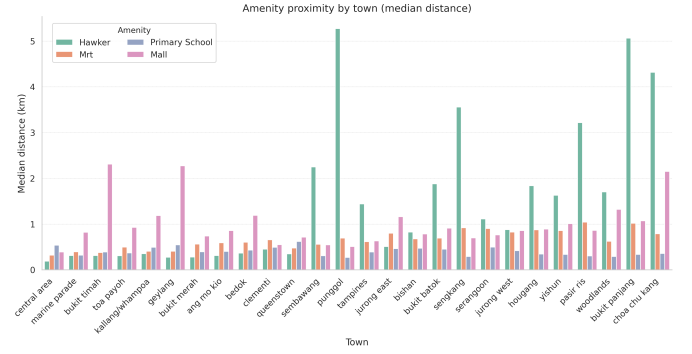


Fig. 11: Median distance (km) to nearest MRT, mall, and primary school by town.

- **Label variants** due to case/whitespace/hyphenation in categoricals (TOWN, STREET, BLOCK, FLOOR_RANGE, FLAT_MODEL, FLAT_TYPE).
- **Heterogeneous floor-range formats** (e.g., “01–3”, “1–3”) and occasional non-parsable tokens.
- **Constant or near-constant fields** in training (e.g., ECO_CATEGORY) and **exact duplicates** within/across splits.
- **Missing or sparse coordinates** affecting amenity-distance computation.

The corrections will be detailed in next section.

III. DATA PREPROCESSING

We prepare the data with a rule-based preprocessing pipeline that yields a single analysis-ready feature dataset for training and test with the same schema. The pipeline standardizes field names and categorical text, enriches each record with a time index, and removes duplicates from training while preventing information leakage. Each transform is deterministic and applied in the same way to both train and test splits. The design follows observations from exploratory analysis, including fragmented categories and temporal movement in prices.

A. Schema and Text Standardization

We apply three deterministic transforms identically to training and test so that models and analysis scripts see one consistent layout.

- **Column-name normalization.** We unify the lease-commencement field by renaming LEASE_COMMENCE_DATA to LEASE_COMMENCE_DATE. This creates a single, consistent field for date logic across datasets and prevents downstream code and joins from failing due to mismatched column names.
- **Case and whitespace normalization for targeted text.** For TOWN, FLAT_TYPE, BLOCK, STREET, FLOOR_RANGE, FLAT_MODEL, and ECO_CATEGORY, we convert text to lower-case and strip leading and trailing spaces. This collapses trivial variants such as stray spaces and case differences that inflated category counts during exploratory analysis and stabilizes the frequencies used by categorical encoders.

c. **Canonicalization of `FLAT_TYPE`.** To remove duplicates created by spacing and hyphenation, we enforce one hyphenated token per numeric flat type. *Mappings:*

```
1 room, 1-room → 1-room
2 room, 2-room → 2-room
3 room, 3-room → 3-room
4 room, 4-room → 4-room
5 room, 5-room → 5-room
```

Labels such as “executive” and “multi generation” remain distinct. The intent is that each genuine flat type is represented by a single label, while unfamiliar labels remain visible for review.

B. Temporal Enrichment (Quarter and RPI)

We attach a macroeconomic housing index to each transaction to capture market conditions at the time of sale. From the recorded month we compute `sale_year` and `sale_quarter` with $q = \lceil \text{month}/3 \rceil$, then left-join the Resale Price Index (RPI) on the year and quarter keys. The lookup is fixed and uses only information available at the sale date. Training and test cover the same calendar span, so no extrapolation is required and no rows are dropped. Temporary parsing fields are removed after the join to keep the schema compact.

C. Constant-Feature Pruning

Using the training split, we test for constant features—columns with zero variance that add no predictive signal yet expand encoders and consume memory. The decision is mirrored to the test split so schemas remain aligned without using test information. In our run, `ECO_CATEGORY` is constant in training and is removed from both training and test. This reduces memory use and avoids redundant indicators.

D. Duplicate Handling (Training Only)

Careful checks found **121** exact duplicates within the training split (removed), **124** within the test split (left unchanged), and **851** records identical across the two splits. Because duplicate rows overweight repeated cases and can inflate validation scores when identical records span folds, we remove exact duplicates from training after standardization. The test split remains unchanged to respect competition rules and preserve its native structure. We acknowledge that identical records across splits constitute train–test data leakage and can bias evaluation; however, as instructed by Professor Chris, we modify only the training data.

E. Leakage Control and Split Isolation

Training and test are provided separately. We process them independently and never combine them. Decisions that depend on data, such as the constant-feature check, are derived from training and then applied to test so that both splits share the same schema without importing test information. *Note on categorical handling in models:* CatBoost uses its built-in ordered target statistics for

categorical features, while XGBoost uses label encoding without target information.

F. Processing Order

The notebook applies the following order: (i) schema and text standardization, including canonicalization of `FLAT_TYPE`; (ii) temporal enrichment by deriving year and quarter and joining the RPI; (iii) constant-feature pruning based on training, mirrored to test; (iv) training-only duplicate removal. This order stabilizes keys and categories before enrichment, finalizes the feature set after enrichment, and removes repeated records just before modeling so the training data are clean and non-duplicated.

G. Summary

TABLE I: Preprocessing diagnostics

Metric	Value
Train (canonical) rows	162,464
Train used in CV (excl. duplicates)	162,343
Test rows	50,000
Duplicates detected in train	121 (0.07%)
Duplicates detected in test	124
Final export: rows dropped from train	121
Constant features dropped	<code>ECO_CATEGORY</code>
RPI join coverage	100% (no rows dropped)
Missing values after preprocessing	none
Train–test identical records	851

The key outcomes are summarised in Table I. The canonical training table remains at 162 464 rows, but CV/fitting uses 162 343 rows after excluding 121 exact duplicates. `ECO_CATEGORY` is dropped as a constant feature. The quarter-level RPI join covers all records, and no missing values remain. Together, these steps standardise schema and text, add an explicit macro index at the sale quarter, and produce one analysis-ready feature dataset for training and test with identical schema.

IV. BASELINE MODELING

The baselines establish reference error levels and reveal which types of information contribute most to accuracy before feature engineering and model tuning.

A. Evaluation protocol

We use a simple 80/20 hold-out split with a fixed seed (`random_state=42`). Inputs are the post-preprocessing, *pre-feature-engineering* columns. Missing numeric values are filled by the median computed on the training fold; in this dataset the four numeric fields are complete, so the fill is a safety guard. The metric is Root Mean Squared Error (RMSE) in SGD.

B. Baseline Models

a. **RF–N (numeric only).** Random Forest with `n_estimators=300` and default settings trained on four raw numeric features: $\{\text{FLOOR_AREA_SQM},$

LEASE_COMMENCE_DATE, rpi, MAX_FLOOR}. This serves as the strict baseline for later comparisons.

- b. **RF-NC (numeric plus low-card categoricals)**. Same learner and split, augmented with one-hot indicators for low-cardinality categoricals such as TOWN, FLAT_TYPE, FLAT_MODEL and FLOOR_RANGE. High-cardinality fields such as STREET and BLOCK are intentionally excluded to keep the baseline compact.

C. Results

TABLE II: Baseline performance on 80/20 hold-out (validation split)

Model	Feature set	RMSE	Δ vs RF-N
RF-N	FLOOR_AREA_SQM, LEASE_COMMENCE_DATE, rpi, MAX_FLOOR	61,036.14	–
RF-NC	RF-N + one-hot TOWN, FLAT_TYPE, FLAT_MODEL, FLOOR_RANGE	36,795.99	–24,240.15

The move from RF-N to RF-NC reduces RMSE by about 24.2k, (see Table II), showing that even shallow categorical segmentation by town, type, model and floor band explains a large share of variance that numeric-only inputs cannot capture.

D. What the baseline models reveal

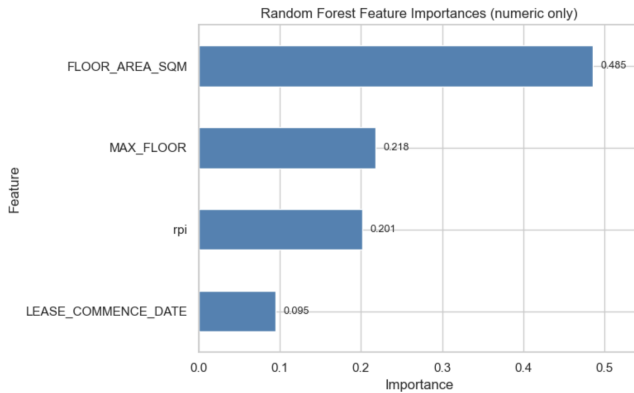


Fig. 12: RF-N numeric-only feature importances. FLOOR_AREA_SQM dominates; MAX_FLOOR is second; rpi is third; LEASE_COMMENCE_DATE is weakest

- **Size dominates among raw numerics.** In RF-N, Fig. 12 shows FLOOR_AREA_SQM as the top feature, MAX_FLOOR second and rpi third, with LEASE_COMMENCE_DATE lowest. This motivates replacing the commencement year with lease-aware features in the next section.
- **Categorical segmentation is critical.** In RF-NC the raw importance ranking in Fig. 13 still places FLOOR_AREA_SQM at the top and rpi next. Individual category indicators such as TOWN_* and FLAT_MODEL_* have small weights on their own, yet taken together they account for the ≈ 24 k RMSE reduction in Table II.

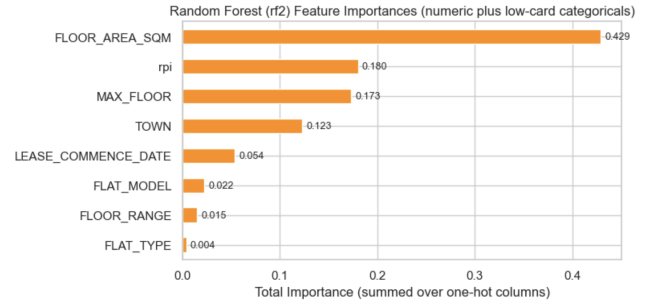


Fig. 13: RF-NC numeric plus low-cardinality categoricals. Size and market level remain strongest; individual category terms such as TOWN_* and FLAT_MODEL_* contribute smaller but additive gains

- **Scope is intentionally narrow.** No hyperparameter tuning is applied; high-cardinality fields and engineered geospatial distances are omitted. This keeps the baseline conservative and makes the gains from feature engineering easy to quantify.

V. FEATURE ENGINEERING

This section describes how raw HDB attributes such as dates, text fields, coordinates, block tags, amenity locations are transformed into model-ready signals. Features are grouped into temporal, proximity, derived floor and block signals, frequency encodings, interactions, and model-specific preparation. Each transformation is motivated by EDA, baseline models and by how tree ensembles discover splits.

A. Temporal Features

Calendar fields are converted into monotonic, comparable scalars that reflect market trend and lease value.

- **SALE_YEAR** and **SALE_MONTH**: extracted from MONTH parsed as YYYY-MM. These give coarse time markers for policy windows and seasonal effects.
- **MONTH_IDX**: integer index counted from the earliest month in the data, which lets tree models capture long-run drift without forcing a seasonal shape.

$$\text{MONTH_IDX} = 12(y - y_0) + (m - m_0),$$

- **FLAT_AGE**: Years elapsed from the lease's commencement year to the year of sale (i.e., how old the flat is at the transaction date).

$$\text{FLAT_AGE} = \text{SALE_YEAR} - \text{LEASE_COMMENCE_DATE}.$$

- **REMAINING_LEASE**: Years left on a standard 99-year HDB lease at the time of sale, truncated at zero so it never goes negative.

$$\text{REMAINING_LEASE} = \max(0, 99 - \text{FLAT_AGE}).$$

SALE_YEAR and SALE_MONTH act as coarse policy/season flags, while MONTH_IDX provides a smooth, monotonic timeline for market drift; together they anchor time effects

that `FLAT_AGE` and `REMAINING_LEASE` cannot capture on their own. `FLAT_AGE` and `REMAINING_LEASE` separate lease value from the sale date and align with how buyers and lenders assess depreciation and bankability, such as CPF and loan eligibility considerations.

B. Proximity Features

Coordinates are converted into distances to key amenities using a `BallTree` with the haversine metric; outputs are in kilometres. Only existing MRT stations are considered; future or planned stations are omitted to avoid forward-looking leakage and reflect accessibility at sale date.

- `distance_to_nearest_hawker`
- `distance_to_nearest_mrt`
- `distance_to_nearest_primary_school`
- `distance_to_nearest_mall`

These proximities turn latitude and longitude into accessibility signals that correlate with price.

C. Derived Floor and Block Features

We parse text fields to obtain comparable numeric signals.

- From `FLOOR_RANGE` (e.g., “04 to 06”), we extract: `FLOOR_MIN`, `FLOOR_MAX`, and `FLOOR_MEDIAN`.
- From `FLAT_TYPE` we derive: `ROOMS` (numeric from the “#-room” pattern), `IS_EXECUTIVE` (indicator).
- `AREA_PER_ROOM = FLOOR_AREA_SQM` divided by `ROOMS`, capturing spaciousness beyond total area.
- From `BLOCK` we derive: `BLOCK_NUM` (numeric part) and `BLOCK_HAS_SUFFIX` (letter suffix flag).

D. Frequency Encodings

To reflect address prevalence without one-hot expansion we add counts based on the combined train+test vocabulary, with no target information used.

- `STREET_COUNT`: frequency of the street string in the full dataset.
- `BLOCK_COUNT`: frequency of the block string in the full dataset.

E. Categorical Codes and Model-Specific Prep

We use the same categorical variables across models; only their representation differs. See Table III for a side-by-side summary that matches our implementation.

a) CatBoost path. We pass *integer-coded* categorical twins with a `*_cat` suffix via `cat_features` (e.g., `MONTH_cat`, `TOWN_cat`, `FLAT_TYPE_cat`, `STREET_cat`, `FLOOR_RANGE_cat`, `FLAT_MODEL_cat`, and interactions such as `TOWN_X_FLAT_TYPE_cat`). Although these values are integers, CatBoost treats the listed columns as *categorical* and converts them using native *ordered target statistics* computed on random permutations that use only earlier rows (leak-safe, smoothed, robust to rare/unseen labels).

b) XGBoost path. XGBoost consumes numeric inputs. We therefore *label-encode* each original categorical column (including interactions) to integer IDs *without* target information; trees split on these codes to form category groups.

This avoids one-hot expansion and keeps dimensionality compact.

TABLE III: Categorical treatment by model (aligned with our code)

Model	Input columns	How categories are turned into numbers
CatBoost	integer-coded <code>*_cat</code> columns listed in <code>cat_features</code> (e.g., <code>MONTH_cat</code> , <code>TOWN_cat</code> , <code>FLAT_TYPE_cat</code> , <code>STREET_cat</code> , <code>FLOOR_RANGE_cat</code> , <code>FLAT_MODEL_cat</code> , <code>TOWN_X_FLAT_TYPE_cat</code>)	Native <i>ordered target statistics</i> on random row permutations using only earlier rows; leak-safe and robust to rare/unseen labels.
XGBoost	label-encoded integer IDs of the <i>original</i> categorical columns (including interactions; no <code>*_cat</code> suffix)	Pandas label encoders (fit on train+test vocabulary, no targets). Trees split over the codes to group categories; avoids one-hot expansion.

Note. One-hot encoding appears only in the **RF-NC** baseline to keep that baseline simple. The `*_cat` variables are used only by CatBoost; XGBoost uses separate label-encoded code columns derived from the original categoricals.

F. Feature Interactions

- **Numeric** `FLOOR_AREA_X_RPI` equals `FLOOR_AREA_SQM` multiplied by `rpi`. This lets the value of space rise or fall with the market level and improves fit in hot or cool periods.
- **Categorical double** `TOWN_X_FLAT_TYPE` joins town and flat type as one token. It captures that the same flat type commands different premia across towns, allowing the model to isolate town-specific size effects with shallow splits (e.g., early, low-depth decision in a tree).
- **Categorical triple** `TOWN_X_FLAT_TYPE_X_MODEL` adds block design to the previous cross. Preferences for specific models differ by town and type; this key exposes that three-way pattern directly.
- **Categorical with vintage tag** `FLAT_TYPE_X_LEASE` joins `FLAT_TYPE` with `LEASE_COMMENCE_DATE` to mark cohorts. It distinguishes older and newer batch within the same type and helps the model separate design vintage from size.

TABLE IV: Final engineered features and rationale

Feature group	Examples and rationale
Temporal	SALE_YEAR, SALE_MONTH, MONTH_IDX, FLAT_AGE, REMAINING_LEASE — trend and lease value on a common scale.
Proximity	distance_to_nearest_hawker, distance_to_nearest_mrt, distance_to_nearest_primary_school, distance_to_nearest_mall — accessibility from coordinates via haversine.
Derived — floor and block	FLOOR_MIN, FLOOR_MAX, FLOOR_MEDIAN; ROOMS, IS_EXECUTIVE, AREA_PER_ROOM; BLOCK_NUM, BLOCK_HAS_SUFFIX.
Frequency	STREET_COUNT, BLOCK_COUNT — address prevalence without one-hot.
Interactions	FLOOR_AREA_X_RPI; TOWN_X_FLAT_TYPE; TOWN_X_FLAT_TYPE_X_MODEL; FLAT_TYPE_X_LEASE.
Model-specific prep	CatBoost — *_cat integer-coded categoricals listed in cat_features; CatBoost applies ordered target statistics (leak-safe). XGBoost — original categorical columns are label-encoded to integer IDs and split by thresholds; this avoids one-hot. (We did not enable XGBoost’s native categorical feature handling.)

VI. MODELING APPROACH

We model resale price with two gradient-boosted tree families well-suited to non-linear, mixed-type tabular data: **CatBoost** [3] and **XGBoost** [4]. The choice follows the baselines (Section IV), which showed large gains once categorical and temporal signals were exposed.

A. Data split and evaluation metric

All experiments use five-fold shuffled KFold cross-validation with a fixed seed (random_state=42). The primary metric is Root Mean Squared Error (RMSE) in SGD; we also report distribution-robust summaries (e.g., median absolute error) in the evaluation section to complement mean-based metrics. To mitigate temporal leakage, all derived time variables use only contemporaneous information (e.g., quarter-level RPI), and preprocessing is applied independently to train and test.

B. Feature set and categorical representation

Both learners use the same engineered feature set in Table IV (see Section V for construction and Table III for categorical specifics). Representation differs only by learner: CatBoost consumes integer-coded *_cat columns via cat_features (ordered target statistics), whereas XGBoost ingests label-encoded integers on the original categoricals (no one-hot).

C. Hyperparameter tuning

We optimise with **Optuna** [5] using 5-fold CV per trial and early stopping where supported. Search spaces (pilot-tested for stability) are:

TABLE V: Model parameters and Optuna search ranges

CatBoost	
Parameter	Range / value
loss_function	RMSE
iterations	[1700, 2100] with early stopping (100)
learning_rate	[0.03, 0.07]
depth	{9, ..., 13}
l2_leaf_reg	[3.5, 5.5]
min_data_in_leaf	[12, 20]
bagging_temperature	[0.7, 1.1]
random_strength	[0.8, 1.5]
subsample	[0.60, 0.65]
colsample_bylevel	[0.85, 0.92]
XGBoost	
Parameter	Range / value
objective	reg:squarederror
eval_metric	rmse
tree_method	hist
n_estimators	[800, 2000] with early stopping (100)
max_depth	{5, ..., 13}
learning_rate	[0.01, 0.15]
subsample	[0.6, 1.0]
colsample_bytree	[0.7, 1.0]
reg_lambda	[0.1, 10.0]
min_child_weight	[1, 20]

D. Training and inference

For each learner we retain the best trial by mean CV RMSE, refit on the full training set, and produce test predictions. We store out-of-fold (OOF) predictions for fair comparisons and potential ensembling. All seeds and folds are fixed for reproducibility.

E. Why this final feature set?

Guided by EDA and many ablations, we end up only keeping these features that improved CV RMSE without leakage:

- **Temporal:** SALE_YEAR and MONTH_IDX capture long-run drift and policy windows using only contemporaneous information from MONTH.
- **Size & lease:** FLOOR_AREA_SQM and REMAINING_LEASE (plus raw LEASE_COMMENCE_DATE to capture cohort/vintage). We omit FLAT_AGE to avoid perfect collinearity with REMAINING_LEASE (99 – age).
- **Market level:** quarterly rpi joined from MONTH absorbs macro trend; FLOOR_AREA_X_RPI lets the value of space vary with market level.
- **Location & access:** town geography via TOWN and distance-to-nearest features distance_to_nearest_mrt, ..., _hawker, ..., _mall; distances are scale-comparable and avoid town-size confounding.

- **Floor & block:** binned `FLOOR_RANGE` (handled as categorical) captures storey premiums; `BLOCK_NUM` supplies block-level signal with minimal cardinality.
- **Targeted interactions:** `TOWN_X_FLAT_TYPE`, `TOWN_X_FLAT_TYPE_X_MODEL`, `FLAT_TYPE_X_LEASE` expose cross-effects seen in EDA; `FLOOR_AREA_X_RPI` already noted above.
- **Exclusions:** no one-hot for high-card fields; no count encodings (`STREET_COUNT`, `BLOCK_COUNT`); no primary-school distance in the final set (low marginal gain, higher collinearity); no polynomial/standardisation (trees are scale-invariant); no forward-looking amenities/policy (leak-safe).

VII. RESULTS AND DISCUSSION

A. Overall performance

Table VI compares cross-validated RMSE across our baseline and boosted models. CatBoost and XGBoost—trained on the same engineered feature set—both more than halve error relative to the numeric-only baseline. A simple 0.5/0.5 blend yields the best score.

TABLE VI: Model performance (5-fold CV). RMSE in SGD.

Model	CV RMSE	Features used	Tuning
Random Forest (Baseline)	61,036.14	4 numeric only	None
CatBoost (Tuned)	25,114.99	20 (numeric + categorical)	Optuna (50 trials)
XGBoost (Tuned)	25,381.50	20 (label-encoded)	Optuna (50 trials)
Ensemble (0.5/0.5)	24,673.13	CatBoost + XGBoost	N/A

Improvement over baseline: 59.58% reduction in RMSE (61,036.1396 $\rightarrow$ 24,673.1310).

B. Cross-validation robustness

CatBoost early-stops at $\sim 2,079$ iterations. Fold RMSEs: {25,079.72, 25,127.79, 25,102.59, 25,294.78, 25,088.80} with $\sigma = 79.69$.

XGBoost is similarly stable: ({25,405.90, 25,449.76, 25,311.62, 25,477.81, 25,261.76}) with $\sigma = 82.16$.

Both standard deviations are $\approx 0.3\%$ of the mean RMSE, indicating a robust protocol. removes the paragraph indent so the lines align flush left.

C. What drives price

Figure 14 contrasts CatBoost and XGBoost importances and their weighted blend. Three patterns emerge:

- **Size $\times$ market level dominates.** `FLOOR_AREA_X_RPI` is the top driver (ensemble $\approx 17.5\%$), capturing that the value of space co-moves with the macro index.
- **Lease vintage matters.** `FLAT_TYPE_X_LEASE`, `LEASE_COMMENCE_DATE` and `REMAINING_LEASE` together account for a large share of importance, reflecting HDB’s 99-year lease structure and financing constraints.

- **Location/type interactions are critical.**

`TOWN_X_FLAT_TYPE` and `TOWN_X_FLAT_TYPE_X_MODEL` explain cross-town premia within the same flat types; base `TOWN`, `FLOOR_RANGE`, `FLAT_MODEL` add incremental signal. Amenity distances (MRT, hawker, mall) contribute smaller but non-negligible effects.

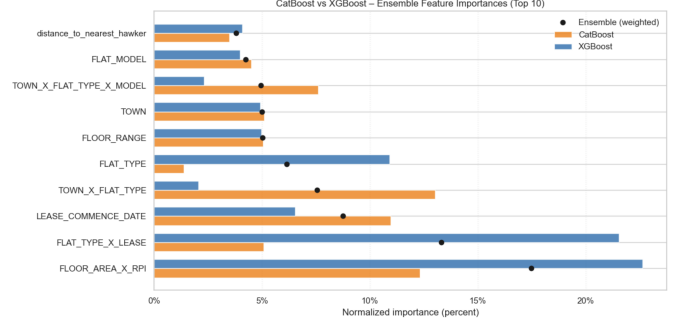


Fig. 14: CatBoost vs. XGBoost feature importances and their weighted ensemble. Interactions and lease-aware variables dominate.

D. Error analysis

We profile absolute and percentage errors by *price band* and by *flat type*. Errors grow with price in absolute terms (expected with RMSE) while percentage errors remain relatively stable.

TABLE VII: Errors by price range (train OOF)

Price bin	MAE	MedAE	Mean%	Med%	N
<300K	12,477.87	9,702.05	4.77	3.67	13,479
300–500K	14,155.67	10,875.85	3.52	2.72	72,930
500–700K	18,442.47	14,157.55	3.13	2.42	50,957
700–900K	24,924.14	19,134.17	3.17	2.44	18,972
>900K	36,627.07	27,056.20	3.55	2.70	6,126

TABLE VIII: Errors by flat type (train OOF)

Flat type	MAE	MedAE	Mean%	Med%	N
multi generation	38,247.94	28,527.07	4.29	3.45	68
executive	26,799.81	20,373.91	3.69	2.94	11,794
5-room	20,330.72	15,087.86	3.31	2.56	40,034
4-room	16,442.58	12,263.67	3.23	2.47	68,740
3-room	13,978.38	10,628.12	3.94	2.98	38,572
1-room	11,089.88	8,513.27	5.38	3.81	56
2-room	10,911.81	8,610.79	3.83	2.95	3,200

Takeaways. (i) Absolute errors increase at the top end ($> 900K$), while percentage errors remain roughly 3–4% across bands; the highest mean percent error occurs in the $< 300K$ bin (small denominators). (ii) *Executive* and *multi-generation* flats have the largest MAE; rare *1-room* sales show the highest *percent* error (not MAE), reflecting scarcity and heterogeneous designs. (iii) The worst individual misses (Appendix, “Top-20”) are premium/rare configurations—adjoined flat, maisonette, DBSS,

terrace—suggesting a single global model underfits this segment.

E. Why the ensemble wins

CatBoost assigns higher weight to categorical interactions (e.g., `TOWN_X_FLAT_TYPE`, `FLAT_TYPE_X_LEASE`), while XGBoost emphasizes smooth numeric interactions (e.g., `FLOOR_AREA_X_RPI`) and large continuous effects (see Fig. 14). A simple 0.5/0.5 average reduces each model’s idiosyncratic errors and yields the best RMSE without extra complexity.

F. Limitations and next steps

- **Premium tail under-representation.** Errors are highest for >900K sales and rare types. A lightweight fix is a two-expert system (one model for >900K), or a quantile/Huber loss to de-emphasize outliers.
- **Missing “premium” signals.** Views, renovation quality, and micro-location (CBD/park/sea) are not captured. Proxies such as CBD distance, waterfront/park buffers, and top-school catchments could narrow the gap.
- **Monotonicity and calibration.** Enforcing monotone constraints for `REMAINING_LEASE` and `FLOOR_AREA_SQM` (supported in XGBoost) and adding isotonic calibration on OOF predictions can improve interpretability and price-level accuracy.

REFERENCES

- [1] C. von der Weth, “Cs5228-2510 project,” <https://kaggle.com/competitions/cs-5228-2510-project>, 2025, kaggle.
- [2] E. Lee, V. Lim, and J. Teo, “Managing the housing market in singapore,” *Economics In Practice: Evidence-based Policymaking In Singapore*, p. 261, 2021.
- [3] A. V. Dorogush, V. Ershov, and A. Gulin, “Catboost: gradient boosting with categorical features support,” *arXiv preprint arXiv:1810.11363*, 2018.
- [4] T. Chen and C. Guestrin, “Xgboost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 785–794.
- [5] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, 2019, pp. 2623–2631.